Stepping up NTFS Forensics: Computing Normal NTFS Timestamp Structures (long version)

Lorenz HORNUNG [a,b,1], Jesse HEYNINCK [a,c] and Harm VAN BEEK [a,b]
[a] *Open Universiteit, the Netherlands*
[b] *Netherlands Forensic Institute, the Netherlands*
[c] *University of Cape Town, South Africa*
ORCiD ID: Lorenz Hornung https://orcid.org/0009-0001-0349-2691, Jesse Heyninck https://orcid.org/0000-0002-3825-4052, Harm van Beek https://orcid.org/0000-0002-6128-5201

Abstract. This is an extended version of

> Lorenz Hornung, Jesse Heyninck, and Harm Beek. Stepping up NTFS Forensics: Computing Normal NTFS Timestamp Structures.

Timestamps of digital entities are a valuable source for criminal investigations, but possible manipulations question their authenticity. To confirm their authenticity, an understanding of expected file timestamps for regular user behavior is needed. This paper addresses this challenge for Windows' New Technology File System (NTFS). Under a restriction on compositions of NTFS operations, we formally identify 4,128 *normal* and 704 *quasi-normal* timestamp structures; i.e., internal relations among timestamps NTFS assigns to files under regular user behavior. To compute these, we provide an Answer Set Programming (ASP) implementation. Further, we provide early empirical results on their prevalence in disk images.

Keywords. digital forensics science, NTFS forensics, timestamp, formal model, Answer Set Programming

Preliminary Remark

This is an extended version of [1] which will be presented at (JURIX 2026). This version provides proofs omitted from the conference version. For citation, please cite the proceedings version once available:

> Lorenz Hornung, Jesse Heyninck, and Harm Beek. Stepping up NTFS Forensics: Computing Normal NTFS Timestamp Structures.

This work is published under the CC BY-NC 4.0 license.

[1] Corresponding Author: Lorenz Hornung, lorenz.hornung@ou.nl

1. Introduction

Law enforcement increasingly relies on digital investigations. One objective of such investigations is to understand which actions have been performed on a device and potentially in what order [2]. For example, investigators may want to establish whether instructions for making a Molotov cocktail were downloaded to a suspect's device before or after they allegedly committed an arson attack. Similarly, they may want to determine whether malicious files were accessed before or after a cyber attack was registered.

Timestamps are an important source of information for such investigative reasoning, as they provide temporal information about when an event was registered on a digital system. A file system, for instance, tracks when a file was created or when its content was last modified [3,4,5].

Importantly, the authenticity of such file timestamps can be questioned, as clock manipulations or anti-forensics tools can be used to manipulate them. For example, the timestamps of downloaded instructions for making a Molotov cocktail could be made to appear as if they were downloaded only after the arson attack. Further, unusual operating system or file system configurations can lead to misleading timestamps as well [6,7].

To assess a timestamp's authenticity is thus a critical step in forensic investigation, especially to confirm or falsify scenarios when manipulation is in play. In this paper, we focus on the assessments of this nature for the New Technology File System (NTFS). NTFS is the default file system for Windows-based operating systems (i.e., Windows' devices) and Windows Servers [8]. Due to the high distribution of Windows' devices, NTFS plays a pivotal role in forensic investigations and digital forensic science [4].

Previous approaches (for an overview, see [9]) assess NTFS timestamps based on empirical observation. An important subset of these observations relies on the internal relations of files' timestamps [10,11,12], which we formally defined in [13]. For NTFS, we refer to these internal relations as *NTFS timestamp structure*.

Contribution. Our contribution lies in identifying 4,128 *normal* and 704 *quasi-normal* NTFS timestamp structures under a restriction on compositions of NTFS operations. These structures represent timestamp relations expected under regular user behavior, with quasi-normal structures obtained by disregarding last-access timestamps. For this identification, we provide a formal framework and computational implementation using Answer Set Programming (ASP). Further, we provide early empirical results on the proportion of these timestamp structures on example disk images.

Outline. First, we present our framework [13] for timestamp structures and operations affecting them (Section 2.1), and specify NTFS operations and (quasi-)normal NTFS timestamp structures (Section 2.2). Having characterized the compositions of NTFS operations requiring checks for the identification of (quasi-)normal timestamp structures under our restriction (Section 3.1), we describe how we use Answer Set Programming (ASP) [14] to compute these timestamp structures (Section 3.2). Lastly, we evaluate disk images against these identified (quasi-)normal NTFS timestamp structures and discuss the impact of these results for forensic science (Section 4).

2. Formal Framework for Identifying Normal NTFS Timestamp Structures

NTFS provides for each file eight timestamps which are retrievable from the Master File Table (MFT). These timestamps are stored as two pairs of four timestamps in the attributes `$STANDARD_INFORMATION` (SI) and `$FILE_NAME` (FN). The timestamps in both SI and FN are labeled as last update time, last access time, last metadata change time, and creation time. Operations on files, such as renaming or copying a file, affect these timestamps. For more information on NTFS file timestamps and its operations, see [12,15,16].

2.1. NTFS File Timestamps, Timestamp Structures, and Transformations

The proposed formalism defining temporal features, data points, timestamp structures and transformations for NTFS is an adaption of our general framework [13].[2]

Temporal features. NTFS, like other file systems, provides timestamps in an ordered fashion by assigning temporal values to predefined variables, which we refer to as *temporal features*. It considers eight temporal features, specified by the following set of temporal features $T := \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3, \mathrm{SI}_4, \mathrm{FN}_1, \mathrm{FN}_2, \mathrm{FN}_3, \mathrm{FN}_4\}$. The temporal features in $\mathrm{SI} = \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3, \mathrm{SI}_4\}$ are the temporal features underlying the timestamps in SI, while those in $\mathrm{FN} = \{\mathrm{FN}_1, \mathrm{FN}_2, \mathrm{FN}_3, \mathrm{FN}_4\}$ underlie the timestamps in FN.

Temporal values. For each file, NTFS assigns temporal values to the temporal features considered. These temporal values are given as numeric values representing a point in time measured as units of 100 nanoseconds that happened after 1601-01-01 00:00:00 UTC. Analogously, temporal values are modeled as positive natural numbers $\mathbb{N}^+$.

Data points. We refer to the assignment of temporal values to temporal features as a *NTFS data point*. Formally speaking, a *NTFS data point* d on T is a function $d : T \to \mathbb{N}^+$. Thus, $d(x)$ provides the timestamp for temporal feature $x \in T$ of data point d.

Timestamp structures. Since $\leq$ imposes a total order on temporal values, data points induce a total preorder on the set of considered temporal features. Based on this, we define NTFS timestamp structures as follows.

Definition 1. *Any total preorder $\preceq$ on T is an NTFS timestamp structure.*

Further, we call any total preorder on $Y \subseteq T$ a restricted NTFS timestamp structure and refer to (restricted) NTFS timestamp structures also as (restricted) timestamp structures. Further, we use $\npreceq$ as the negation, $\prec$ as the strict part, and $=$ as the equality of a timestamp structure $\preceq$. Besides $\preceq$, we use R to refer to NTFS timestamp structures with $\preceq_R = R$. Lastly, $\mathscr{R}_T$ denotes the set of all NTFS timestamp structures.

Transformations. Within NTFS, several operations can be performed on files that affect file timestamps and hence timestamp structures. Since not all NTFS operations will be modeled by their effect on timestamp structures (see below), we call these operations affecting timestamp structures *transformations* to avoid ambiguity. These transformations might not result in a unique timestamp structure, but a set of timestamp structures.

[2] In [13], we define timestamps, timestamp structures, and the transformations (called operations) that affect them independent of a file system (and hence independent of a fixed set of temporal features). Their notion of timestamp structure is defined relative to data points. The adapted formalism employed here is conceptually equivalent, yet it omits the formal grounding of timestamp structures in those data points.

Let $s : T \to \text{SI}$ be defined as $s(\text{SI}_\text{i}) = \text{SI}_\text{i}$ and $s(\text{FN}_\text{i}) = \text{SI}_\text{i}$. For any $R_i \in \mathscr{R}_T$,

$\mathsf{AC}(R_i) = \{R_i\}$

$\mathsf{AT}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T : x \preceq_{R_j} \text{SI}_3 \wedge R_i \restriction_{T \setminus \{\text{SI}_3\}} = R_j \restriction_{T \setminus \{\text{SI}_3\}}\}$

$\mathsf{UP}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T \setminus \{\text{SI}_2\} : x \preceq_{R_j} \text{SI}_3 \preceq_{R_j} \text{SI}_2 \wedge R_i \restriction_{T \setminus \{\text{SI}_2,\text{SI}_3\}} = R_j \restriction_{T \setminus \{\text{SI}_2,\text{SI}_3\}}\}$

$\mathsf{OC}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T : x \preceq_{R_j} \text{SI}_3 \wedge R_j \restriction_{T \setminus \{\text{SI}_2,\text{SI}_3\}} = R_i \restriction_{T \setminus \{\text{SI}_2,\text{SI}_3\}}\}$

$\mathsf{OV}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T : x \preceq_{R_j} \text{SI}_3 \wedge R_j \restriction_{T \setminus \{\text{SI}_1,\text{SI}_2,\text{SI}_3\}} = R_i \restriction_{T \setminus \{\text{SI}_1,\text{SI}_2,\text{SI}_3\}}\}$

$\mathsf{RE}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T : x \preceq_{R_j} \text{SI}_3 \wedge \forall x, y \in T \setminus \{\text{SI}_3\} : x \preceq_{R_j} y \iff s(x) \preceq_{R_i} s(y)\}$

Figure 1. Characterization of NTFS operations AC, AT, UP, OC, OV, RE as NTFS transformations

Definition 2. *An NTFS transformation is a function* $G : \mathscr{R}_T \to (\mathscr{P}(\mathscr{R}_T) \setminus \emptyset)$.

Given $X \subseteq T$, $G(R_i)\restriction_X$ denotes the restriction of every timestamp structure $R_j \in G(R_i)$ to X: $G(R_i)\restriction_X = \{R_j\restriction_X \mid R_j \in G(R_i)\}$ where for all $x, y \in X$, $x \preceq_{R_j\restriction_X} y \iff x \preceq_{R_j} y$.

Transformations can be applied subsequently forming a composition of transformations. A composition of transformations $G_2 \odot G_1 : \mathscr{R}_X \to (\mathscr{P}(\mathscr{R}_T) \setminus \emptyset)$ is defined as for all $R_i \in \mathscr{R}_T$, $(G_2 \odot G_1)(R_i) = \{R_k \in \mathscr{R}_T \mid \exists R_j \in G_1(R_i)$ such that $R_k \in G_2(R_j)\}$.

2.2. Modeling NTFS Operations

To model NTFS file operations[3], we formalize the empirical characterization by [16]. Importantly, we impose the following assumptions: (1) the operation end time is greater than or equal to the operation start time, and (2) operation start time and end time are greater than or equal to any temporal value. As a consequence, if a temporal feature is assigned operation start time or end time due to an operation, then no temporal feature not assigned start or end time is ranked higher in the timestamp structure.

2.2.1. NTFS Operations as Initial NTFS Timestamp Structures

We model some NTFS operations as initial NTFS timestamp structures, because they assign (nearly) all temporal features a new temporal value. Thus, they instantiate complete timestamp structures. The operations modeled as timestamp structures are `Create` (CR), `Copy` (CO), and `Move from other volume` (MV).

They instantiate timestamp structures as follows. While CR assigns every temporal feature the operation start time ($\text{SI}_1 = \text{SI}_2 = \text{SI}_3 = \text{SI}_4 = \text{FN}_1 = \text{FN}_2 = \text{FN}_3 = \text{FN}_4$), CO preserves the timestamp SI_2 of the copied file, assigns the remaining temporal features, besides SI_3, the starting time, and assigns SI_3 the end time ($\text{SI}_2 \preceq \text{SI}_1 = \text{SI}_4 = \text{FN}_1 = \text{FN}_2 = \text{FN}_3 = \text{FN}_4 \preceq \text{SI}_3$). Lastly, MV keeps the timestamps of SI_1 and SI_2 unchanged, sets SI_3 to the operation end time, and assigns the operation start time to all remaining temporal features (resulting in $\text{SI}_1 \preceq \text{SI}_2 \preceq \text{SI}_4 = \text{FN}_1 = \text{FN}_2 = \text{FN}_3 = \text{FN}_4 \preceq \text{SI}_3$ or $\text{SI}_2 \preceq \text{SI}_1 \preceq \text{SI}_4 = \text{FN}_1 = \text{FN}_2 = \text{FN}_3 = \text{FN}_4 \preceq \text{SI}_3$). Summarizing, these operations yield the following set of initial NTFS timestamp structures $\mathscr{R}_\text{init}$.

[3]Readers interested in more information on described NTFS operations are referred to [12,16].

$$\mathscr{R}_{\text{init}} = \{R \in \mathscr{R}_T \mid \text{SI}_1 \preceq_R \text{SI}_2 \preceq_R \text{SI}_4 =_R \text{FN}_1 =_R \text{FN}_2 =_R \text{FN}_3 =_R \text{FN}_4 \preceq_R \text{SI}_3 \text{ or}$$
$$\text{SI}_2 \preceq_R \text{SI}_1 \preceq_R \text{SI}_4 =_R \text{FN}_1 =_R \text{FN}_2 =_R \text{FN}_3 =_R \text{FN}_4 \preceq_R \text{SI}_3\}$$

2.2.2. NTFS Operations as NTFS Transformation

We characterize the following NTFS file operations[4] as NTFS transformations (in sense of Definition 2): `Access` (AC), `Attribute change` (AT), `Update` (UP), `Overwriting copy` (OC), `Overwriting from other volume` (OV), and `Rename` (RE).

These operations are formally defined as NTFS transformations in Figure 1 and express the following impacts. While AC does not affect the timestamp structure, AT assigns the operation start time to SI_3 and UP assigns operation end time to SI_2 and operation start time to SI_3. Transformation OC and OV assign operation start time to SI_3, while keeping the temporal value assigned to SI_2, or SI_1 and SI_2, respectively. As a consequence, these temporal features (SI_2 or SI_1 and SI_2) can take any rank in the timestamp structure except those that are strictly greater than SI_3. Lastly, RE synchronizes the temporal value of FN_1, FN_2, FN_3, and FN_4 with SI_1, SI_2, SI_3, and SI_4, respectively. Further, SI_3 is assigned the operation start time.

2.2.3. Normal NTFS Timestamp Structures

We define normal NTFS timestamps based on a reachability problem: An NTFS timestamp structure is normal if an initial timestamp structure exists alongside a composition of transformations resulting in the structure in question. We define X-normality analogously, if this property holds when restricting the timestamp structure to a subset X of temporal features.

Definition 3. *Given the set of initial NTFS timestamp structures $\mathscr{R}_{init}$, NTFS transformation $\mathscr{G} = \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}, \mathsf{OC}, \mathsf{OV}, \mathsf{RE}\}$, and set of temporal features $X \subseteq T$, we say that $R \in \mathscr{R}$ is*

- *normal, if there exist $R_{init} \in \mathscr{R}_{init}$ and $G_n \odot \cdots \odot G_1$ with $G_1, \ldots, G_n \in \mathscr{G}$ such that $R \in G_n \odot \cdots \odot G_1(R_{init})$,*
- *X-normal, if there exist $R_{init} \in \mathscr{R}_{init}$ and $G_n \odot \cdots \odot G_1$ with $G_1, \ldots, G_n \in \mathscr{G}$ such that $R \restriction_X \in G_n \odot \cdots \odot G_1(R_{init}) \restriction_X$.*

If R is *not* normal, we say that R is abnormal. Analogously, if R is *not* X-normal, we say that R is X-abnormal. X-normality is essential to reason about NTFS timestamp structures when it is not known whether last access is tracked, as discussed in Section 2.2.4.

2.2.4. Remark on Modifiers and NTFS Operations

Bouma et al. [16] discuss modifiers that alter the effect of NTFS operations on NTFS file timestamps and hence on NTFS timestamp structures. Below we briefly describe these modifiers and their treatment within our formalism.

[4] NTFS also provides the operations `Delete` and `Move within volume` which affect timestamp structures identically to `Access` and `Rename`; therefore they are omitted for brevity.

Let $s : T \to \text{SI}$ be defined as $s(\text{SI}_\text{i}) = \text{SI}_\text{i}$ and $s(\text{FN}_\text{i}) = \text{SI}_\text{i}$. For any $R_i \in \mathscr{R}_T$,
$\mathsf{FT}(R_i) = \{R_j \in \mathscr{R}_T \mid \forall x \in T : x \preceq_{R_j} \text{SI}_3 \wedge \forall x, y \in T \setminus \{\text{SI}_1, \text{SI}_3\} : x \preceq_{R_j} y \iff s(x) \preceq_{R_i} s(y)\}$

Figure 2. Characterization of RE affected by file tunneling as transformation FT

Transfer from FAT and exFAT. File Allocation Table (FAT) and Extensible File Allocation Table (exFAT) are file systems by Windows mostly found on portable storage utilities like USB flash drives. Transfer of files from such sources into an NTFS does not result in a different specification of CO, MV, OC, and OV, as it at most affects the resolution of temporal value assigned to SI_1 and SI_2, but not the resulting timestamp structures.

File tunneling. When a file is deleted and another file is added with the same filename in the same directory within a short time window, the new file inherits the temporal values assigned to SI_1 and FN_1 from the deleted file. This might affect CR, CO, and RE leading to the characterization of CR and CO affected by file tunneling as timestamp structures $\mathscr{R}_{\text{FT}}$ and RE affected by file tunneling as transformation FT specified in Figure 2.

$$\mathscr{R}_{\text{FT}} = \{R \in \mathscr{R}_T \mid \text{FN}_1 \preceq_R \text{SI}_1 \preceq_R \text{SI}_2 =_R \text{SI}_3 =_R \text{SI}_4 =_R \text{FN}_2 =_R \text{FN}_3 =_R \text{FN}_4 \text{ or}$$
$$\text{SI}_1 \preceq_R \text{FN}_1 \preceq_R \text{SI}_2 =_R \text{SI}_3 =_R \text{SI}_4 =_R \text{FN}_2 =_R \text{FN}_3 =_R \text{FN}_4\}$$

Importantly, Proposition 1 implies that we can disregard FT in our search for normal NTFS structures because any timestamp structure that can be obtained via FT can also be reached using the $\mathsf{OV} \odot \mathsf{RE}$.

Proposition 1. *For any $R \in \mathscr{R}_T$, $\mathsf{FT}(R) \subseteq \mathsf{OV} \odot \mathsf{RE}(R)$.*

Proof. Let $R_i \in \mathsf{FT}(R)$ be arbitrary. We need to construct R_j such that $R_j \in \mathsf{RE}(R)$ and $R_i \in \mathsf{OV}(R_j)$. Note that by definition of FT and RE, $\mathsf{FT}(R) \restriction_{T \setminus \{\text{SI}_1\}} = \mathsf{RE}(R) \restriction_{T \setminus \{\text{SI}_1\}}$. Thus, there exist $R'_i \in \mathsf{RE}(R)$ such that $R'_i \restriction_{T \setminus \{\text{SI}_1\}} = R_i \restriction_{T \setminus \{\text{SI}_1\}}$. Let $R_j = R'_i$. We need to show that $R_i \in \mathsf{OV}(R_j)$ by proving that the following conditions hold: (i) for all $x \in T$, $x \preceq_{R_i} \text{SI}_3$; and (ii) $R_i \restriction_{T \setminus \{\text{SI}_1, \text{SI}_2, \text{SI}_3\}} = R_j \restriction_{T \setminus \{\text{SI}_1, \text{SI}_2, \text{SI}_3\}}$. Condition (i) holds due to definition of FT and (ii) holds, as $R_j \restriction_{T \setminus \{\text{SI}_1\}} = R_i \restriction_{T \setminus \{\text{SI}_1\}}$ by construction. □

Last access updating. When last access updating is enabled, a file's last-access timestamp (SI_4) is updated across operations AC, OC, OV (operation start time) and UP, CO, MV (operation end time). The characterizations in Figure 1 thus do not apply to transformations affected by this setting. Although full treatment exceeds the scope of this paper, we can address this implicitly by excluding SI_4 and FN_4 from our timestamp structures. Thus, we consider $T \setminus \{\text{SI}_4, \text{FN}_4\}$-normal, or *quasi-normal*, timestamp structures.[5] We choose this approach as last-access updating is disabled by default.

3. Computing Normal NTFS Timestamp Structures

We first show that some compositions of NTFS transformations are reducible to a known composition with an identical resulting set of NTFS timestamp structures (Section 3.1). Using this result, we compute normal NTFS timestamp structures by applying the re-

[5] We defer complete characterizations to future work.

duced compositions to initial structures, and obtain quasi-normal NTFS timestamp structures by restricting these normal structures to $T \setminus \{\mathrm{SI}_4, \mathrm{FN}_4\}$. For these computations, we provide an implementation based on Answer Set Programming (Section 3.2).

3.1. Reducing Compositions of NTFS Transformations

In the following, we propose a technique to reduce compositions of NTFS transformation without altering the resulting set of timestamp structures and illustrate how this is done for a fragment of them; i.e., the fragment where the `Rename` operation RE occurs at most once.[6] For these reduction, we introduce reduction rules as defined in Definition 4.

Definition 4. *Given some NTFS transformations G_1, G_2 and G, we call $(G_2 \odot G_1) \simeq_X G$ a reduction rule, if (i) $X \subseteq T$, and (ii) for any $R \in \mathscr{R}_T$, $(G_2 \odot G_1)(R)\restriction_X = G(R)\restriction_X$.*

If $X = T$, we call $(G_2 \odot G_1) \simeq_X G$ a *general* reduction rule and otherwise, a *restricted* reduction rule. Further, general reduction rules imply restricted reduction rules, as shown in Proposition 2.

Proposition 2. *Let G_1, G_2, and G be some NTFS transformations and $X \subseteq T$ be a set of NTFS temporal features. If $(G_2 \odot G_1) \simeq_X G$ is a reduction rule, then $(G_2 \odot G_1) \simeq_Y G$ with $Y \subseteq X$ is a reduction rule as well.*

Proof. Suppose that $(G_2 \odot G_1) \simeq_X G$ is a reduction rule. That is, $(G_2 \odot G_1)(R)\restriction_X = G(R)\restriction_X$. It needs to be shown that $R_i \in (G_2 \odot G_1)(R)\restriction_Y \iff R_i \in G(R)\restriction_Y$ with $Y \subseteq X$. For the left-to-right direction, suppose that $R_i \in (G_2 \odot G_1)(R)\restriction_Y$. Since $Y \subseteq X$, there exists $R_j \in (G_2 \odot G_1)(R)\restriction_X$ with $R_j\restriction_Y = R_i$. Since $(G_2 \odot G_1)(R)\restriction_X = G(R)\restriction_X$, there exist $R_k \in G(R)\restriction_X$ with $R_j = R_k$ so that $R_k\restriction_Y = R_i$. This implies that $R_i \in G(R)\restriction_Y$. The right-to-left direction follows by analogous reasoning. □

To obtain such reduction rules for NTFS operations, we will introduce two properties of operations relative to sets of temporal features: *X-depending* and *X-preserving*.

Definition 5. *For any NTFS transformation G, we say that*

- *G is X-depending, if $X \subseteq T$ is the least subset (w.r.t. set inclusion) s.t. for any $R_i, R_j \in \mathscr{R}_T$, $R_i\restriction_X = R_j\restriction_X$ implies that $G(R_i) = G(R_j)$;*
- *G is X-preserving, if $X \subseteq T$ with $|X| > 1$ is the greatest subset (w.r.t. set inclusion) such that for any $R_i \in \mathscr{R}_T$, and any $R_j \in G(R_i)$, $R_i\restriction_X = R_j\restriction_X$.*

We denote by $\mathsf{Dep}(G)$ the subset $X \subseteq T$ for which G is *X-depending*, and by $\mathsf{Pres}(G)$ the subset $Y \subseteq T$ for which G is *Y-preserving*.

Proposition 3. *Table 1 specifies $\mathsf{Dep}(G)$ and $\mathsf{Pres}(G)$ for* AC, AT, UP, RE, OV, *and* OC.

Proof. The proof technique is similar in all cases. Let us show the proof for UP.

We show that $X = T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$ is $\mathsf{Dep}(\mathsf{UP})$; i.e., X is the least subset of T such that for any $R_i, R_j \in \mathscr{R}_T$, if $R_i\restriction_X = R_j\restriction_X$, then $\mathsf{UP}(R_i) = \mathsf{UP}(R_j)$.

First let us show that for any $R_i, R_j \in \mathscr{R}_T$, if $R_i\restriction_X = R_j\restriction_X$, then $\mathsf{UP}(R_i) = \mathsf{UP}(R_j)$. Let $R_i, R_j \in \mathscr{R}_T$ be arbitrary and suppose that $R_i\restriction_X = R_j\restriction_X$. This implies that $\mathsf{UP}(R_i) = \{R_k \in$

[6] We defer reducing remaining NTFS compositions to future work.

Table 1. $\mathsf{Dep}(G)$ and $\mathsf{Pres}(G)$ for $\mathsf{AC}, \mathsf{AT}, \mathsf{UP}, \mathsf{RE}, \mathsf{OV}$, and OC

	AC	AT	UP	RE	OV	OC
$\mathsf{Dep}(G)$	T	$T \setminus \{\mathrm{SI}_3\}$	$T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$	SI	$\mathrm{FN} \cup \{\mathrm{SI}_4\}$	$T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$
$\mathsf{Pres}(G)$	T	$T \setminus \{\mathrm{SI}_3\}$	$T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$	$\mathrm{SI} \setminus \{\mathrm{SI}_3\}$	$\mathrm{FN} \cup \{\mathrm{SI}_4\}$	$T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$

$\mathscr{R}_T \mid \forall x \in T \setminus \{\mathrm{SI}_2\} : x \preceq_{R_k} \mathrm{SI}_3 \preceq_{R_k} \mathrm{SI}_2 \wedge R_i{\restriction_X} = R_k{\restriction_X}\} = \{R_k \in \mathscr{R}_T \mid \forall x \in T \setminus \{\mathrm{SI}_2\} : x \preceq_{R_k} \mathrm{SI}_3 \preceq_{R_k} \mathrm{SI}_2 \wedge R_j{\restriction_X} = R_k{\restriction_X}\} = \mathsf{UP}(R_j)$ by the definition of UP.

Second, let us show that X is the least subset of T satisfying this constraint. It suffices to show that for any $Y \subseteq T$ and any $R_i, R_j \in \mathscr{R}_T$, if $R_i{\restriction_Y} = R_j{\restriction_Y} \Rightarrow \mathsf{UP}(R_i) = \mathsf{UP}(R_j)$, then $X \subseteq Y$. Proof by contraposition. Suppose that $X \not\subseteq Y$, i.e., there exists $x_i \in X$ such that $x_i \notin Y$. It suffices to provide $R, R' \in \mathscr{R}_T$ such that $R{\restriction_Y} = R'{\restriction_Y}$ and $\mathsf{UP}(R) \neq \mathsf{UP}(R')$. Let R be defined as for all $x, y \in T$, $x \preceq_R y$. Further, let R' be defined as $x_i \preceq_{R'} x_i$, and for any $x, y \in T \setminus \{x_i\}$, $x \preceq_{R'} y$ and $x \prec_{R'} x_i$. Clearly, $R{\restriction_Y} = R'{\restriction_Y}$. However, $\mathsf{UP}(R) \neq \mathsf{UP}(R')$, because for all $R_i \in \mathsf{UP}(R)$ and $R_j \in \mathsf{UP}(R')$, $x_i \preceq_{R_i} x_j$ and $x_i \not\preceq_{R_j} x_j$ for any $x_j \in X \setminus \{x_i\}$.

Let us show that $X = T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}$ is $\mathsf{Pres}(\mathsf{UP})$; i.e., X is the greatest subset of T such that for any $R \in \mathscr{R}_T$ and any $R_i \in \mathsf{UP}(R)$, $R{\restriction_X} = R_i{\restriction_X}$. Note that by the definition of UP, it holds that $R{\restriction_X} = R_i{\restriction_X}$ for any $R \in \mathscr{R}_T$ and any $R_i \in \mathsf{UP}(R)$. It remains to show that for any $Y \subseteq T$ with $|Y| > 1$, if for any $R \in \mathscr{R}_T$ and $R_i \in \mathsf{UP}(R)$, $R{\restriction_Y} = R_i{\restriction_Y}$, then $Y \subseteq X$.

Proof by contraposition. Suppose that $Y \not\subseteq X$ with $|Y| > 1$. Then, there exists $x \in Y$ such that $x \notin X$ which implies that $x = \mathrm{SI}_2$ or $x = \mathrm{SI}_3$. It suffices to provide a timestamp structure $R_i \in \mathscr{R}_T$ such that $R_j \in \mathsf{UP}(R_i)$ with $R_i{\restriction_Y} \neq R_j{\restriction_Y}$. Consider the following:

$$\mathrm{SI}_1 = \mathrm{SI}_2 = \mathrm{SI}_3 = \mathrm{SI}_4 = \mathrm{FN}_1 = \mathrm{FN}_2 = \mathrm{FN}_3 = \mathrm{FN}_4 \tag{R_i}$$

so that $R_j \in \mathsf{UP}(R_i)$ with

$$\mathrm{SI}_1 = \mathrm{SI}_4 = \mathrm{FN}_1 = \mathrm{FN}_2 = \mathrm{FN}_3 = \mathrm{FN}_4 \prec \mathrm{SI}_3 \prec \mathrm{SI}_2 \tag{R_j}$$

Since $|Y| > 1$, and $x \in Y$ with $x = \mathrm{SI}_2$ or $x = \mathrm{SI}_3$, (i) $\mathrm{SI}_2, \mathrm{SI}_3 \in Y$, (ii) $\mathrm{SI}_2, y \in Y$, or (iii) $\mathrm{SI}_3, y \in Y$ for some $y \in X$. If (i), then $\mathrm{SI}_2 \preceq_{R_i{\restriction_Y}} \mathrm{SI}_3$ and $\mathrm{SI}_2 \not\preceq_{R_j{\restriction_Y}} \mathrm{SI}_3$, while if (ii), then $\mathrm{SI}_2 \preceq_{R_i{\restriction_Y}} y$ and $\mathrm{SI}_2 \not\preceq_{R_j{\restriction_Y}} y$. If (iii), $\mathrm{SI}_3 \preceq_{R_i{\restriction_Y}} y$ and $\mathrm{SI}_3 \not\preceq_{R_j{\restriction_Y}} y$. Since these cases were exhaustive, it follows that $R_i{\restriction_Y} \neq R_j{\restriction_Y}$. □

Given $\mathsf{Dep}(G)$ and $\mathsf{Pres}(G)$, we can establish an insight underlying reduction rules. Notably, if a transformation preserves all features that a subsequently applied transformation depends on, then the application of the former becomes redundant.

Proposition 4. *Given two NTFS transformations G_1 and G_2 such that $\mathsf{Dep}(G_2) \subseteq \mathsf{Pres}(G_1)$, then for any $R \in \mathscr{R}_T$, $(G_2 \odot G_1)(R) = G_2(R)$.*

Proof. Let $R \in \mathscr{R}_T$ be an arbitrary NTFS timestamp structure, and G_1 and G_2 be two NTFS transformations with $\mathsf{Pres}(G_1) = X$ and $\mathsf{Dep}(G_2) = Y$. Suppose that $Y \subseteq X$. Let $R_i \in G_1(R)$ be arbitrary. Since G_1 is X-preserving, $R_i{\restriction_X} = R{\restriction_X}$. It follows that $R_i{\restriction_Y} = R{\restriction_Y}$ because $Y \subseteq X$. Since G_2 is Y-input-dependent and $R_i{\restriction_Y} = R{\restriction_Y}$, $G_2(R_i) = G_2(R)$. Since $R_i \in G_1(R)$ was arbitrary, it holds that for all $R' \in G_1(R)$, $G_2(R') = G_2(R)$. This means that $(G_2 \odot G_1)(R) = G_2(R)$. □

Propositions 3 and 4 allow to derive the following general reduction rules.

Proposition 5. *General reduction rules for* AC, AT, UP, OC, OV, *and* RE*:*

- $(G \odot \mathsf{AC}) \simeq_T G$ *for* $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{RE}, \mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$
- $(G \odot \mathsf{AT}) \simeq_T G$ *for* $G \in \{\mathsf{AT}, \mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$
- $(G \odot \mathsf{UP}) \simeq_T G$ *for* $G \in \{\mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$
- $(G \odot \mathsf{OC}) \simeq_T G$ *for* $G \in \{\mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$
- $(\mathsf{OV} \odot \mathsf{OV}) \simeq_T \mathsf{OV}$
- $(\mathsf{AC} \odot G) \simeq_T G$ *for* $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{RE}, \mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$
- $(\mathsf{AT} \odot G) \simeq_T G$ *for* $G \in \{\mathsf{AT}, \mathsf{RE}, \mathsf{OC}, \mathsf{OV}\}$
- $(\mathsf{OC} \odot \mathsf{OV}) \simeq_T \mathsf{OV}$

Proof. 1.-5. follow from Propositions 3 and 4, while 6. is implied by the fact that for any $R \in \mathscr{R}_T$, $\mathsf{AC}(R) = \{R\}$.

For 7., let $R \in \mathscr{R}_T$ be arbitrary. For the left-to-right direction, let $R_i \in \mathsf{AT} \odot G(R)$ with $G \in \{\mathsf{AT}, \mathsf{RE}, \mathsf{OC}, \mathsf{OV}\}$ be arbitrary. Hence there exist $R_j \in G(R)$ such that $R_i \in \mathsf{AT}(R_j)$. By the definition of AT, $R_i \restriction_{T \setminus \{\mathrm{SI}_3\}} = R_j \restriction_{T \setminus \{\mathrm{SI}_3\}}$ and for all $x \in T$, $x \preceq_{R_i} \mathrm{SI}_3$. By the definitions of $\mathsf{AT}, \mathsf{RE}, \mathsf{OC}$ and OV this implies that $R_i \in G(R)$. For the right-to-left direction, let $R_i \in G(R)$ with $G \in \{\mathsf{AT}, \mathsf{RE}, \mathsf{OC}, \mathsf{OV}\}$ be arbitrary. By the definitions of $\mathsf{AT}, \mathsf{RE}, \mathsf{OC}$ and OV, $x \in T \setminus \{\mathrm{SI}_3\}$, $x \preceq_{R_i} \mathrm{SI}_3$. By definition of AT it hence follows that $R_i \in \mathsf{AT}(R_i)$.

To prove 8., let $R \in \mathscr{R}_T$ be arbitrary. For the left-to-right direction, let $R_i \in \mathsf{OC} \odot \mathsf{OV}(R)$ be arbitrary. This means that there exists $R_j \in \mathsf{OV}(R)$ such that $R_i \in \mathsf{OC}(R_j)$. To show that $R_i \in \mathsf{OV}(R)$, it suffices to show that (i) $\forall x \in T : x \preceq_{R_i} \mathrm{SI}_3$, and (ii) $R_i \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}} = R \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}}$. Note that (i) holds due to the definition of OC. For (ii), by definition of OC, $R_i \restriction_{T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}} = R_j \restriction_{T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}}$ which implies that $R_i \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}} = R_j \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}}$. Further, the definition of OV implies that $R_j \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}} = R \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}}$. Consequently, $R_i \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}} = R_j \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}} = R \restriction_{T \setminus \{\mathrm{SI}_1, \mathrm{SI}_2, \mathrm{SI}_3\}}$. For the right-to-left direction, let $R_i \in \mathsf{OV}(R)$ be arbitrary. It suffices to show that $R_i \in \mathsf{OC}(R_i)$. By definition of OV, $\forall x \in T : x \preceq_{R_i} \mathrm{SI}_3$, while $R_i \restriction_{T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}} = R_i \restriction_{T \setminus \{\mathrm{SI}_2, \mathrm{SI}_3\}}$ clearly holds. It follows from definition of OC that $R_i \in \mathsf{OC}(R_i)$. □

Proposition 2 implies that for any $X \subseteq T$, the aforementioned rules are restricted reduction rules. For instance, $(\mathsf{UP} \odot \mathsf{OC}) \simeq_{\mathrm{SI}} \mathsf{UP}$.

Next, we show that compositions of NTFS transformations where RE occurs at most once are reducible to one known composition: We reduce the sub-composition from the start until the occurrence of RE (Lemma 1) and the one starting after it (Lemma 2).

Lemma 1. *For any composition of NTFS transformations* $G_n \odot \ldots \odot G_1$ *with* $G_n = \mathsf{RE}$, $G_i \neq \mathsf{RE}$ *for all* $n > i$*, and any* $R \in \mathscr{R}_T$, $G_n \odot \cdots \odot G_1(R) = \varphi(R)$ *with* $\varphi \in \Phi_1$

$$\Phi_1 = \{\mathsf{RE},\ \mathsf{RE} \odot \mathsf{UP},\ \mathsf{RE} \odot \mathsf{AT},\ \mathsf{RE} \odot \mathsf{AT} \odot \mathsf{UP}, \mathsf{RE} \odot \mathsf{OC},$$
$$\mathsf{RE} \odot \mathsf{OV},\ \mathsf{RE} \odot \mathsf{UP} \odot \mathsf{OV},\ \mathsf{RE} \odot \mathsf{AT} \odot \mathsf{UP} \odot \mathsf{OV}\}$$

Proof. Let $G_n \odot \cdots \odot G_1$ be an arbitrary composition of NTFS operations. Suppose that $G_n = \mathsf{RE}$ and $G_i \neq \mathsf{RE}$ for all $n > i$. We need to show that there exists $\varphi \in \Phi_1$ such that $G_n \odot \cdots \odot G_1(R) = \varphi(R)$ for any $R \in \mathscr{R}_T$. Therefore, let $R \in \mathscr{R}_T$ arbitrary and let us construct φ with $\varphi \in \Phi_1$.

Since $\mathsf{Dep}(\mathsf{RE}) = \mathsf{SI}$, it suffices to construct ψ such that $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathsf{SI}} = \psi(R)\restriction_{\mathsf{SI}}$. Then for $\varphi = \mathsf{RE} \odot \psi$, $G_n \odot \cdots \odot G_1(R) = \varphi(R)$. We construct ψ by a case distinction due to the properties of $G_{n-1} \odot \cdots \odot G_1$.

Case 1. OV *occurs in* $G_{n-1} \odot \ldots \odot G_1$.

There exist G_i such that $G_i = \mathsf{OV}$ and for all $G_{i'}$ with $n-1 \geq i' > i$, $G_{i'} \neq \mathsf{OV}$. By Propositions 2 and 5, for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}, \mathsf{OC}, \mathsf{OV}\}$, $\mathsf{OV} \odot G \simeq_{\mathsf{SI}} \mathsf{OV}$. By stepwise applying these reduction rules, $G_i \odot \ldots \odot G_1(R)\restriction_{\mathsf{SI}} = \mathsf{OV}(R)\restriction_{\mathsf{SI}}$. It remains to construct τ such that for any $R' \in \mathscr{R}_T$, $\tau(R')\restriction_{\mathsf{SI}} = G_{n-1} \odot \ldots \odot G_{i+1}(R')\restriction_{\mathsf{SI}}$. Let us consider the following cases.

Case 1.1. UP *occurs in* $G_{n-1} \odot \ldots \odot G_{i+1}$

There exists G_j such that $G_j = \mathsf{UP}$ and for all $G_{j'}$ with $n-1 \geq j' > j$, $G_{j'} \neq \mathsf{UP}$.

Case 1.1.1. OC *occurs in* $G_{n-1} \odot \ldots \odot G_{j+1}$

There exists G_k such that $G_k = \mathsf{OC}$ and for all $G_{k'}$ with $n-1 \geq k' > k$, $G_{k'} \neq \mathsf{OC}$. Propositions 2 and 5 state that for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}, \mathsf{OC}\}$, $\mathsf{OC} \odot G \simeq_{\mathsf{SI}} \mathsf{OC}$. By stepwise applying these reduction rules to $G_k \odot \ldots \odot G_{i+1}$, for any $R' \in \mathscr{R}_T$, $G_k \odot \ldots \odot G_{i+1}(R')\restriction_{\mathsf{SI}} = \mathsf{OC}(R')\restriction_{\mathsf{SI}}$. Further, Propositions 2 and 5 imply that for any $G \in \{\mathsf{AC}, \mathsf{AT}\}$, $G \odot \mathsf{OC} \simeq_{\mathsf{SI}} \mathsf{OC}$. By stepwise applying these reduction rules to $G_{n-1} \odot \ldots \odot G_k$, $G_{n-1} \odot \ldots \odot G_k(R')\restriction_{\mathsf{SI}} = \mathsf{OC}(R')\restriction_{\mathsf{SI}}$ for any $R' \in \mathscr{R}_T$. By Propositions 2 and 5, $\mathsf{OC} \odot \mathsf{OV} \simeq_{\mathsf{SI}} \mathsf{OV}$ and therefore $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathsf{SI}} = \mathsf{OV}(R)\restriction_{\mathsf{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{OV}$ in this case.

Case 1.1.2. OC *does not occur in* $G_{n-1} \odot \ldots \odot G_{j+1}$.

Propositions 2 and 5 imply that for any $G \in \{\mathsf{AC}, \mathsf{AT}, , \mathsf{UP}, \mathsf{OC}\}$, $\mathsf{UP} \odot G \simeq_{\mathsf{SI}} \mathsf{UP}$. By stepwise applying these rules to $G_j \odot \ldots \odot G_{i+1}$, for any $R' \in \mathscr{R}_T$, $G_j \odot \ldots \odot G_{i+1}(R')\restriction_{\mathsf{SI}} = \mathsf{UP}(R')\restriction_{\mathsf{SI}}$. This means that $G_j \odot \ldots \odot G_1(R)\restriction_{\mathsf{SI}} = \mathsf{UP} \odot \mathsf{OV}(R)\restriction_{\mathsf{SI}}$. It remains to reduce $G_{n-1} \odot \ldots \odot G_{j+1}$. Note that only AT, or AC might occur in $G_{n-1} \odot \ldots \odot G_{j+1}$.

(a) AT occurs in $G_{n-1} \odot \ldots \odot G_{j+1}$. Therefore, there exists $G_k = \mathsf{AT}$ such that for all $G_{k'}$ with $n-1 \geq k' > k$, $G_{k'} \neq \mathsf{AT}$. Further, for all $G_{k''}$ with $k > k'' > j$, $G_{k''} = \mathsf{AT}$, or $G_{k''} = \mathsf{AC}$. By Propositions 2 and 5, for any $G \in \{\mathsf{AC}, \mathsf{AT}\}$, $\mathsf{AT} \odot G \simeq_{\mathsf{SI}} \mathsf{AT}$. Therefore, for any $R' \in \mathscr{R}_T$, $G_k \odot \ldots \odot G_{j+1}(R')\restriction_{\mathsf{SI}} = \mathsf{AT}(R')\restriction_{\mathsf{SI}}$. This implies that $G_k \odot \ldots \odot G_1(R)\restriction_{\mathsf{SI}} = \mathsf{AT} \odot \mathsf{UP} \odot \mathsf{OV}(R)\restriction_{\mathsf{SI}}$. Further note that for all $G_{k'}$ with $n-1 \geq k' > k$, $G_{k'} = \mathsf{AC}$. By Proposition 2 and 5, $\mathsf{AC} \odot \mathsf{AT} \simeq_{\mathsf{SI}} \mathsf{AT}$. By stepwise applying this rule to $G_{n-1} \odot \ldots \odot G_k$, $G_{n-1} \odot \ldots \odot G_k(R')\restriction_{\mathsf{SI}} = \mathsf{AT}(R')\restriction_{\mathsf{SI}}$ for any $R' \in \mathscr{R}_T$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{AT} \odot \mathsf{UP} \odot \mathsf{OV}$ in this case.

(b) AT does not occur in $G_{n-1} \odot \ldots \odot G_{j+1}$. Therefore, for all G_k with $n-1 \geq k > j$, $G_k = \mathsf{AC}$. By Proposition 2 and 5, $\mathsf{AC} \odot \mathsf{UP} \simeq_{\mathsf{SI}} \mathsf{UP}$. By stepwise applying this rule to $G_{n-1} \odot \ldots \odot G_j$, $G_{n-1} \odot \ldots \odot G_j(R')\restriction_{\mathsf{SI}} = \mathsf{UP}(R')\restriction_{\mathsf{SI}}$ for any $R' \in \mathscr{R}_T$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{UP} \odot \mathsf{OV}$ in this case.

Case 1.2. UP *does not occur in* $G_{n-1} \odot \ldots \odot G_{i+1}$.

By Propositions 2 and 5, for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{OC}\}$, $G \odot \mathsf{OV} \simeq_{\mathrm{SI}} \mathsf{OV}$. By stepwise applying these reduction rules to $G_{n-1} \odot \ldots \odot G_i$, $G_{n-1} \odot \ldots \odot G_i(R')\restriction_{\mathrm{SI}} = \mathsf{OV}(R')\restriction_{\mathrm{SI}}$ for any $R' \in \mathscr{R}_T$. Therefore, $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{OV}(R)\restriction_{\mathrm{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{OV}$ in this case.

Case 2. OV *doesn't occur in* $G_{n-1} \odot \ldots \odot G_1$*, but* OC *does.*

There exist G_i such that $G_i = \mathsf{OC}$ and for all $G_{i'}$ with $n-1 \geq i' > i$, $G_{i'} \neq \mathsf{OC}$. By Propositions 2 and 5, for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}, \mathsf{OC}\}$, $\mathsf{OC} \odot G \simeq_{\mathrm{SI}} \mathsf{OC}$. By stepwise applying these reduction rules to $G_i \odot \ldots \odot G_1$, $G_i \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{OC}(R)\restriction_{\mathrm{SI}}$. It remains to construct τ such that for any $R' \in \mathscr{R}_T$, $\tau(R')\restriction_{\mathrm{SI}} = G_{n-1} \odot \ldots \odot G_{i+1}(R')\restriction_{\mathrm{SI}}$. Let us consider the following cases.

Case 2.1. UP *occurs in* $G_{n-1} \odot \ldots \odot G_{i+1}$.

There exist G_j such that $G_j = \mathsf{UP}$ and for all $G_{j'}$ with $n-1 \geq j' > j$, $G_{j'} \neq \mathsf{UP}$. Propositions 2 and 5 imply that for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}\}$, $\mathsf{UP} \odot G \simeq_{\mathrm{SI}} \mathsf{UP}$. By stepwise applying these rules to $G_j \odot \ldots \odot G_{i+1}$, for any $R' \in \mathscr{R}_T$, $G_j \odot \ldots \odot G_{i+1}(R')\restriction_{\mathrm{SI}} = \mathsf{UP}(R')\restriction_{\mathrm{SI}}$. This means that $G_j \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{UP} \odot \mathsf{OC}(R)\restriction_{\mathrm{SI}}$. Since $\mathsf{UP} \odot \mathsf{OC} \simeq_{\mathrm{SI}} \mathsf{UP}$ due to Propositions 2 and 5, it follows that $G_j \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. It remains to reduce $G_{n-1} \odot \ldots \odot G_{j+1}$. Note that only AT or AC might occur in $G_{n-1} \odot \ldots \odot G_{j+1}$.

(a) If AT occurs in $G_{n-1} \odot \ldots \odot G_{j+1}$, then by analogous reasoning to 1.1.2(a) one obtains that $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{AT} \odot \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{AT} \odot \mathsf{UP}$ in this case.

(b) If AT does not occur in $G_{n-1} \odot \ldots \odot G_{j+1}$, then by analogous reasoning to 1.1.2(b) one obtains that $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{UP}$ in this case.

Case 2.2. UP *does not occur in* $G_{n-1} \odot \ldots \odot G_{i+1}$.

By analogous reasoning to 1.2, $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{OC}(R)\restriction_{\mathrm{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{OC}$.

Case 3. *Neither* OV *nor* OC *occur in* $G_{n-1} \odot \ldots \odot G_1$.

Case 3.1. UP *occurs in* $G_{n-1} \odot \ldots \odot G_1$.

There exists $G_i = \mathsf{UP}$ such that for all $G_{i'}$ with $n-1 \geq i' > i$, $G_{i'} \neq \mathsf{UP}$. By Propositions 2 and 5, for any $G \in \{\mathsf{AC}, \mathsf{AT}, \mathsf{UP}\}$, $\mathsf{UP} \odot G \simeq_{\mathrm{SI}} \mathsf{UP}$. This implies that $G_i \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. If AT occurs in $G_{n-1} \odot \ldots \odot G_{i+1}$, then by analogous reasoning to 1.1.2(a), $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{AT} \odot \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. Therefore, $\varphi = \mathsf{RE} \odot \mathsf{AT} \odot \mathsf{UP}$ in this case. In contrast, if AT does not occur in $G_{n-1} \odot \ldots \odot G_{i+1}$, then by analogous reasoning to 1.1.2(b), $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{UP}(R)\restriction_{\mathrm{SI}}$. Consequently, $\varphi = \mathsf{RE} \odot \mathsf{UP}$ in this case.

Case 3.2. UP *does not occur in* $G_{n-1} \odot \ldots \odot G_1$.

(a) If AT occurs in $G_{n-1} \odot \ldots \odot G_1$, then by analogous reasoning to 1.1.2(a), it follows that $G_{n-1} \odot \ldots \odot G_1(R)\restriction_{\mathrm{SI}} = \mathsf{AT}(R)\restriction_{\mathrm{SI}}$. Consequently, $\phi = \mathsf{RE} \odot \mathsf{AT}$ in this case.

(b) If AT does not occur in $G_{n-1} \odot \ldots \odot G_1$, then $G_n \odot \ldots \odot G_1 = \mathsf{RE} \odot \mathsf{AC} \odot \ldots \odot \mathsf{AC}$ (if $n > 1$), or $G_n \odot \ldots \odot G_1 = \mathsf{RE}$ (if $n = 1$). If the former is the case, then for all G_i with $n > i \geq 1$, $G_i = \mathsf{AC}$. By Proposition 5, $\mathsf{AC} \odot \mathsf{AC} \simeq_{\mathrm{SI}} \mathsf{AC}$. This implies that $G_{n-1} \odot \ldots \odot G_1(R) \restriction_{\mathrm{SI}} = \mathsf{AC}(R) \restriction_{\mathrm{SI}}$. Since $\mathsf{RE} \odot \mathsf{AC} \simeq_{\mathrm{SI}} \mathsf{RE}$ by Proposition 5, $\varphi = \mathsf{RE}$ in this case. Clearly, if the later is the case, $\varphi = \mathsf{RE}$.

□

Lemma 2. *For any composition of NTFS transformations $G_n \odot \ldots \odot G_1$ with $G_i \neq \mathsf{RE}$ for $n \geq i \geq 1$ and any $R \in \mathscr{R}_T$, $G_n \odot \cdots \odot G_1(R) = \varphi(R)$ with $\varphi \in \Phi_2$*

$$\Phi_2 = \{\mathsf{AC},\ \mathsf{UP},\ \mathsf{AT},\ \mathsf{AT} \odot \mathsf{UP},\ \mathsf{OC},\ \mathsf{OV},\ \mathsf{UP} \odot \mathsf{OV},\ \mathsf{AT} \odot \mathsf{UP} \odot \mathsf{OV}\}$$

Proof. By analogous reasoning to proof of Lemma 1 based on the reduction rules given in Proposition 5. □

This implies Theorem 1: The effect of any composition of NTFS transformations where RE occurs at most once is identical to the effect of one of 80 known compositions.

Theorem 1. *For any composition of NTFS operations $G_n \odot \cdots \odot G_1$ with $|\{1 \leq i \leq n \mid G_i = \mathsf{RE}\}| \leq 1$, and any $R \in \mathscr{R}_T$, $G_n \odot \cdots \odot G_1(R) = \varphi(R)$ with $\phi \in \Phi$*

$$\Phi := \Phi_1 \cup \Phi_2 \cup \{\tau \odot \psi \mid \psi \in \Phi_1 \text{ and } \tau \in \Phi_2\}$$

Proof. If RE does not occur in $G_n \odot \cdots \odot G_1$ or $G_n = \mathsf{RE}$, then Lemmas 1 and 2 imply that there exist $\varphi \in \Phi_1$ or $\varphi \in \Phi_2$ such that $G_n \odot \cdots \odot G_1(R) = \varphi(R)$.

It remains to consider the case where RE occurs in $G_n \odot \cdots \odot G_1$ and $G_n \neq \mathsf{RE}$. In this case, there exist G_i such that $G_i = \mathsf{RE}$ and for all $G_{i'}$ with $n \geq i' > i$, $G_{i'} \neq \mathsf{RE}$. By Lemma 1, there exists $\psi \in \Phi_1$ such that $G_i \odot \ldots \odot G_1(R) = \psi(R)$. Since RE does not occur in $G_n \odot \ldots \odot G_{i+1}$, Lemma 2 implies that there exists τ such that for any $R' \in \mathscr{R}_T$, $G_n \odot \ldots \odot G_{i+1}(R') = \tau(R')$. This implies that $G_n \odot \ldots \odot G_1(R) = \tau \odot \psi(R)$. □

3.2. Answer Set Programming Implementation

Theorem 1 allows us to compute *some* normal NTFS timestamp structures efficiently, as we only need to apply the 80 compositions specified to the initial NTFS timestamp structures (see Section 2.2). This does not guarantee that *all* normal NTFS timestamp structures are identified because of the restriction on the NTFS compositions (i.e., RE occurs at most once). We provide an Answer Set Programming (ASP) implementation using clingo [14] that performs these computations and hence identifies these normal timestamp structures.[7] Readers who are unfamiliar with ASP are referred to [18].

Sketch of implementation. We define an ASP program that computes, for an initial NTFS timestamp structure $R \in \mathscr{R}_{\text{init}}$ and a composition of NTFS transformations $\varphi \in \Phi$, the set of timestamp structures $\varphi(R)$. To encode the (initial) NTFS timestamp structures, NTFS transformations and compositions of them, two predicates are of interest: `leq(S,X,Y)` and `apply(A,O)`. `leq(S,X,Y)` allows us to characterize a timestamp structure in state `S` and denotes that `X` $\preceq$ `Y` in `S`. By specifying `leq(0,X,Y)`, we de-

[7] Implementation will be publicly accessible and is included in the supplementary material [17].

fine initial NTFS timestamp structures. Further, we can define the effect of transformations by characterizing `leq(S+1,X,Y)` if the transformation is applied to state `S`. If a transformation yields multiple outputs, like `Overwriting Copy`, we use choice rules to obtain multiple answer sets with distinct timestamp structures defined by different `leq(S+1,X,Y)`. `apply(S,A)` indicates that transformation `A` is applied in state `S`, yielding state `S+1`. This allows for defining compositions of NTFS transformations. Given a composition of NTFS transformations defined by `apply(0,A).` ... `apply(i,A').`, facts of the form `leq(i+1,X,Y)` specify a timestamp structure obtained by applying the composition to an initial timestamp structure. Note that different resulting timestamp structures are given by different answer sets. We then extract these resulting timestamp structures. Using the python clingo API, this is done for any $(\varphi, R) \in \Phi \times \mathscr{R}_{\text{init}}$ yielding normal NTFS timestamp structures which are then converted into a human-readable format. Lastly, we restrict these normal NTFS timestamp structures to $T \setminus \{\text{SI}_4, \text{FN}_4\}$ to obtain quasi-normal ($T \setminus \{\text{SI}_4, \text{FN}_4\}$-normal) NTFS timestamp structures.

Computed normal NTFS timestamp structures. Under the current restriction on compositions, we identify 4,128 normal timestamp structures and 704 quasi-normal timestamp structures. Our implementation provides a list of these identified structures.[8] After computing the resulting timestamp structures for $(\varphi, R) \in \Phi \times \mathscr{R}_{\text{FT}}$ and $(\varphi, R) \in \Phi \times \{R^*\}$ with $R^* := T \times T$ modeling the NTFS operation CR, we can show that taking file tunneling $\mathscr{R}_{\text{FT}}$ or R^* as the initial timestamp structure(s) yield the same set of (quasi-)normal timestamp structures:

Proposition 6. *The following holds for normal NTFS timestamp structures:*

$$\begin{aligned}\{R \in \mathscr{R}_T \mid R \textit{ is normal}\} \supseteq \{R \in \mathscr{R}_T \mid \exists (\varphi, R') \in \Phi \times \mathscr{R}_{init} : R \in \varphi(R')\} \\ = \{R \in \mathscr{R}_T \mid \exists (\varphi, R') \in \Phi \times \mathscr{R}_{FT} : R \in \varphi(R')\} \\ = \{R \in \mathscr{R}_T \mid \exists (\varphi, R') \in \Phi \times \{R^*\} : R \in \varphi(R')\}\end{aligned}$$

Proof. Follows from the identified timestamp structures for each initial structure under the restrictions. They can be found in [17]. □

4. Normal NTFS Timestamp Structures in Practice

To assess the usability of the identified set of (quasi-)normal NTFS timestamp structures for forensic investigations, we evaluate their proportion across ten NTFS disk images (see Table 2) that were constructed for capture-the-flag (CTF) or training purposes. We applied no specific criteria for this selection. We sketch our implementation for evaluating the proportion of these (quasi-)normal timestamp structures on a disk image, before presenting the results and discussing their implications for digital forensics.[9]

[8] By the Fubini number [19], it follows that the identified set of (quasi-)normal timestamp structures are 0.76% of 545,835 possible NTFS timestamp structure and 15.03% of 4,683 possible NTFS timestamp structures restricted to $T \setminus \{\text{SI}_4, \text{FN}_4\}$.

[9] Implementation and results will be publicly accessible and are included in the supplementary material [17].

Sketch of implementation. The evaluation requires only the MFT from a disk image, where NTFS timestamps are stored. It is extractable with various forensic software; in our case, we use Hansken [20,21]. We convert the MFT to JSON with MFTECmd [22] which parses and extracts file metadata including timestamps. We process each file's metadata specified in the JSON, deduce its NTFS timestamp structure, evaluate whether its timestamp structure is identical to an identified (quasi-)normal timestamp structure, and store this information in a csv file.

Results. We find 1,903,105 NTFS timestamp structures in total, of which 460,760 (24.21%) match a normal timestamp structure identified under the current restriction, and 1,838,994 (96.63%) match a computed quasi-normal one under the current restriction.

Table 2. Proportion of files matching (quasi-)normal timestamp structures under the current restriction.

	Avatar [23]	Badguy [24]	CC-EN [25]	CC-NL [25]	Data Leakage [26]
comp. normal	18.20%	33.97%	13.68%	24.72%	17.27%
comp. quasi-normal	86.07%	77.88%	99.03%	99.03%	95.71%

	Hacking [27]	Hunter [28]	İGÜCTF24 [29]	Schiphol [30]	TechHive [31]
comp. normal	24%	22.56%	8.05%	55.81%	21.92%
comp. quasi-normal	96.40%	90.87%	97.81%	98.85%	98.17%

Discussion. It is expected that timestamp structures found will not match the identified normal nor quasi-normal timestamp structures, as we did not compute all possible ones due to restriction on compositions (RE occurs at most once). Hence, a non-match does not by itself establish that a timestamp structure is (quasi-)abnormal. This restriction might also explain the significant difference in percentage of files with matching identified quasi-normal and matching identified normal timestamp structures, but the definition of RE (e.g., file rename) makes this less plausible. Alternatively, it could be explained if NTFS disk images have been configured with last-access updating turned on or alternative operations not described in forensic literature [16] impact last-access timestamps.

Implications for digital forensics. The high percentage of computed quasi-normal timestamp structures suggests that (quasi-)normality can serve as a criterion for assessing authenticity of timestamps. For a file, (quasi-)normality indicates that its timestamps are explainable by NTFS operations. At the disk level, a benchmark for expected percentage of (quasi-)normal timestamp structures can be established. Timestamps from a disk with a significantly lower percentage might be considered less authentic. However, such a benchmark requires computing (quasi-)normal timestamp structures for all compositions, including those with multiple occurrences of RE (e.g., multiple file renames), and conducting empirical tests using real hard drive images.

5. Conclusion and Discussion

As digitization advances, digital investigations are essential to law enforcement. Consequently, digital forensic science bears a significant responsibility for upholding accuracy and accountability. To fulfill this responsibility, rigorous techniques are of importance.

Summary. We contribute to this challenge by proposing a formal mechanism for computing (quasi-)normal NTFS timestamps; i.e., timestamp structures expected for regular user behavior. To this end, we model NTFS operations to obtain starting timestamp structures, as well as transformations that affect these structures (Section 2.2). Further, we show that some compositions of NTFS transformations where RE occurs at most once are reducible to a defined set of compositions (Section 3.1). Based on these restricted compositions, we identify 4,128 normal and 704 quasi-normal NTFS timestamp structures using an implementation based on ASP (Section 3.2). Further, we provide a tool to analyze timestamp structures of disk images and investigate timestamp structures found on multiple disk images. Our results suggest that (quasi-)normal timestamp structures might provide insights for forensic investigations, pending future work (Section 4).

Related work. To our knowledge, (quasi-)normal NTFS timestamp structures have not yet been formally studied. Nevertheless, our work relates to research in forensic science and legal reasoning. First, timestamp structures, which we formally defined in [13], have been studied empirically to derive heuristics for (i) identifying timestamp tampering [11,12], (ii) previously performed NTFS operations [16], or (iii) previously used applications [32]. Second, Answer Set Programming (ASP) has proven effective for modeling legal reasoning [33,34,35,36] and digital forensic reasoning [37,38,39]. We advance this research line by applying ASP to file system forensics and NTFS forensics.

Limitations and future work. Our work faces three main limitations in order to obtain a technique usable in digital investigations. First, we identified some (quasi-)normal NTFS timestamp structures, but potentially not all. To overcome this limitation, one needs to lift the restriction that NTFS operations modeled by RE, such as file renaming, occurs at most once in compositions of transformations. If these can be reduced to a known set of compositions, one can then use the technique proposed to identify these potentially missing timestamp structures (as described in Section 3.2). Second, a complete characterization of NTFS operations affected by last-access updating is missing. Ideally, resolving this shortcoming should result in a higher proportion of normal timestamp structures for disk images, when they already have a high proportion of quasi-normal timestamp structures. Third, testing of the formalism against more realistic disk images is essential. Combining this with detailed analysis of acquired data about normal timestamp structures (e.g., file type, file path or system configurations), would result in a more detailed understanding of (ab-) normal NTFS timestamp structures in practice. Empirical testing should also compare the model against validated ground truth to identify false positives (empirically abnormal timestamps structures classified as normal) and false negatives (empirically normal timestamps structures classified as abnormal).

References

[1] Hornung L, Heyninck J, Van Beek H. Stepping up NTFS Forensics: Computing Normal NTFS Timestamp Structures. In: Legal Knowledge and Information Systems (JURIX 2026); 2026. .

[2] Breitinger F, Studiawan H, Hargreaves C. SoK: Timeline Based Event Reconstruction for Digital Forensics: Terminology, Methodology, and Current Challenges. Forensic Science International: Digital Investigation. 2025 Jul;53:301932.

[3] Buchholz F, Spafford E. On the role of file system metadata in digital forensics. Digital Investigation. 2004;1(4):298-309.

[4] Carrier B. File system forensic analysis. 16th ed. Addison-Wesley; 2017.

[5] Casey E. Digital evidence and computer crime: forensic science, computers and the Internet. 3rd ed. Academic Press; 2011.

[6] Conlan K, Baggili I, Breitinger F. Anti-forensics: Furthering digital forensic science through a new extended, granular taxonomy. Digital Investigation. 2016;18:S66-75.

[7] Neale C. Fool Me Once: A Systematic Review of Techniques to Authenticate Digital Artefacts. Forensic Science International: Digital Investigation. 2023 Jun;45:301516.

[8] Microsoft. NTFS Overview. Microsoft; 2026. Accessed: August 25, 2026. Available from: `https://learn.microsoft.com/en-us/windows-server/storage/file-server/ntfs-overview`.

[9] Oh J, Lee S, Hwang H. Forensic Detection of Timestamp Manipulation for Digital Forensic Investigation. IEEE Access. 2024;12:72544-65.

[10] Ding X, Zou H. Time based data forensic and cross-reference analysis. In: Proceedings of the 2011 ACM Symposium on Applied Computing. SAC '11. New York, NY, USA: Association for Computing Machinery; 2011. p. 185–190.

[11] Cho GS. A computer forensic method for detecting timestamp forgery in NTFS. Computers & Security. 2013;34:36-46.

[12] Galhuber M, Luh R. Time for Truth: Forensic Analysis of NTFS Timestamps. In: Proceedings of the 16th International Conference on Availability, Reliability and Security. Vienna Austria: ACM; 2021. p. 1-10.

[13] Hornung L, Jonker H, Van Beek H. Connecting File Timestamps: A Formal Approach. In: Proceedings of the Digital Forensics Doctoral Symposium (DFDS '26), March 24, 2026, Linköping, Sweden. New York, NY, USA: Association for Computing Machinery; 2026. .

[14] Gebser M, Kaminski R, Kaufmann B, Schaub T. Multi-shot ASP solving with clingo. Theory and Practice of Logic Programming. 2019;19(1):27-82.

[15] Microsoft. File Times. Microsoft; 2026. Accessed: August 25, 2026. Available from: `https://learn.microsoft.com/en-us/windows/win32/sysinfo/file-times`.

[16] Bouma J, Jonker H, Van Der Meer V, Van Den Aker E. Reconstructing Timelines: From NTFS Timestamps to File Histories. In: Proceedings of the 18th International Conference on Availability, Reliability and Security. Benevento Italy: ACM; 2023. p. 1-9.

[17] Hornung L, Heyninck J, van Beek H. Supplementary Material for "Stepping up NTFS Forensics: Computing Normal NTFS Timestamp Structures"; 2026. Includes an extended version with proofs and implementation code. `https://edu.nl/3e8ru`.

[18] Gebser M, Kaminski R, Kaufmann B, Schaub T. Answer Set Solving in Practice. Springer International Publishing; 2013.

[19] OEIS Foundation Inc. A000670: Fubini numbers; 2026. Accessed: August 25, 2026. `https://oeis.org/A000670`.

[20] Netherlands Forensic Institute. Hansken Digital Evidence Platform. Netherlands Forensic Institute; 2026. Accessed: August 25, 2026. Available from: `https://www.hansken.nl/`.

[21] van Beek HMA, van Eijk EJ, van Baar RB, Ugen M, Bodde JNC, Siemelink AJ. Digital forensics as a service: Game on. Digital Investigation. 2015;15:20-38.

[22] Zimmerman E. MFTECmd; 2026. Accessed: August 25, 2026. Available from: `https://github.com/EricZimmerman/MFTECmd`.

[23] Guidance Software. Avatar Benchmark Image. Guidance Software; 2010. Proprietary Forensic Training Disk Image, EnCase Training Curriculum.

[24] Okonkwo U. Forensics Image Test Image. National Institute of Standards and Technology (NIST); 2022. Accessed: August 25, 2026. Available from: `https://cfreds.nist.gov/all/DFIR_AB/ForensicsImageTestimage`.

[25] Henseler H, Van Beek H. ChatGPT as a Copilot for Investigating Digital Evidence. In: Proceedings of the Third International Workshop on Artificial Intelligence and Intelligent Assistance for Legal Professionals in the Digital Workplace (LegalAIIA 2023); 2023. CC-EN and CC-NL datasets are available for research purposes upon request from the authors. Available from: https://ceur-ws.org/Vol-3423/paper6.pdf.

[26] National Institute of Standards and Technology (NIST). Data Leakage Case. National Institute of Standards and Technology (NIST); 2018. Accessed: August 25, 2026. Available from: https://cfreds-archive.nist.gov/data_leakage_case/data-leakage-case.html.

[27] National Institute of Standards and Technology (NIST). Hacking Case. National Institute of Standards and Technology (NIST); 2018. Accessed: August 25, 2026. Available from: https://cfreds-archive.nist.gov/Hacking_Case.html.

[28] CyberDefenders. Hunter Lab. CyberDefenders; 2026. Accessed: August 25, 2026. Available from: https://cyberdefenders.org/blueteam-ctf-challenges/hunter/.

[29] Demir M. İGÜCTF24 Forensics. National Institute of Standards and Technology (NIST); 2024. Accessed: August 25, 2026. Available from: https://cfreds.nist.gov/all/G%CTF24'(MehmetDemir)/GCTF24Forensics.

[30] Meconi T. Pattern-of-Life Forensics in Hansken. Hogeschool van Amsterdam; 2021. Bachelor thesis. Schiphol dataset is available for research purposes upon request from the author.

[31] Spencer M. The TechHive Scenario. National Institute of Standards and Technology (NIST); 2026. Accessed: August 25, 2026. Available from: https://cfreds.nist.gov/all/MarkSpencer/TheTechHiveScenario.

[32] Galhuber M, Luh R. Timestamp-Based Application Fingerprinting in NTFS. In: Proceedings of the 19th International Conference on Availability, Reliability and Security. Vienna Austria: ACM; 2024. p. 1-10.

[33] Aravanis T, Demiris K, Peppas P. Legal Reasoning in Answer Set Programming. In: 2018 IEEE 30th International Conference on Tools with Artificial Intelligence (ICTAI). IEEE; 2018. p. 302-6.

[34] Arias J, Moreno-Rebato M, Rodriguez-García JA, Ossowski S. Automated legal reasoning with discretion to act using s(LAW). Artificial Intelligence and Law. 2024;32(4):1141-64.

[35] Theseider Dupré D. Answer Set Programming for Legal Decision Support and Explanation. In: Sileno G, Spanakis J, Van Dijck G, editors. Frontiers in Artificial Intelligence and Applications. IOS Press; 2023. p. 383-6.

[36] Dik J. Explaining Discretionary Judicial Decisions Using Answer Set Programming. In: Markovich R, Di Caro L, Rapp A, Schifanella C, editors. Frontiers in Artificial Intelligence and Applications. IOS Press; 2025. p. 49-60.

[37] Costantini S, De Gasperis G, Olivieri R. Digital Forensics Evidence Analysis: An Answer Set Programming Approach for Generating Investigation Hypotheses. In: Calimeri F, Ianni G, Truszczynski M, editors. Logic Programming and Nonmonotonic Reasoning. vol. 9345. Springer International Publishing; 2015. p. 242-9.

[38] Costantini S, De Gasperis G, Olivieri R. Digital forensics and investigations meet artificial intelligence. Annals of Mathematics and Artificial Intelligence. 2019;86(1-3):193-229.

[39] Lisi FA, Sterlicchio G. Mining Sequences in Phone Recordings with Answer Set Programming. In: Proceedings of HYDRA-RCRA 2022: 1st International Workshop on HYbrid Models for Coupling Deductive and Inductive ReAsoning and 29th RCRA Workshop on Experimental Evaluation of Algorithms for Solving Problems with Combinatorial Explosion; 2022. Available from: https://ceur-ws.org/Vol-3281/paper4.pdf.